Guide-Lessons1

Claude Code Persistence Guide

CLAUDE.MD | RULES | MEMORY.MD

A practical reference for how Claude Code loads, chains, and prioritizes instructions across sessions and projects.

---

1. CLAUDE.MD

What It Is

`CLAUDE.md` is a markdown file you write to give Claude Code standing instructions. It loads automatically at session start — no prompting needed.

The Hierarchy

Claude Code walks UP the directory tree from your working directory, loading every `CLAUDE.md` it finds. All files merge into one combined context.

```
SCOPE                PATH
EXCLUDABLE?
_________________    ____________________________________________________
___________
Managed Policy       /Library/Application Support/ClaudeCode/
NO (never)
User Level           ~/.claude/CLAUDE.md
Yes
Ancestor Dirs        ~/Documents/CLAUDE.md
Yes
Project Level        ~/Documents/myproject/CLAUDE.md
Yes
                     ~/Documents/myproject/.claude/CLAUDE.md
Yes
```

```
Subdirectory       ~/Documents/myproject/src/CLAUDE.md
Yes (lazy)
```

ASCII Diagram — Loading Flow

```
Session starts in ~/Documents/myproject/src/


┌──────────────────────────────────────────────────────┐
│  /Library/Application Support/ClaudeCode/            │ ◄── Org pol
icy (always)
│  CLAUDE.md                                           │
├──────────────────────────────────────────────────────┤
│  ~/.claude/CLAUDE.md                                 │ ◄── Your pe
rsonal prefs
├──────────────────────────────────────────────────────┤
│  ~/CLAUDE.md                                         │ ◄── Ancesto
r walk
│  ~/Documents/CLAUDE.md                               │ ◄── Ancesto
r walk
├──────────────────────────────────────────────────────┤
│  ~/Documents/myproject/CLAUDE.md                     │ ◄── Project
root
│  ~/Documents/myproject/.claude/CLAUDE.md             │ ◄── Alt loc
ation
├──────────────────────────────────────────────────────┤
│  ~/Documents/myproject/src/CLAUDE.md                 │ ◄── Lazy (o
n demand)
└──────────────────────────────────────────────────────┘

      ▲ All loaded eagerly at startup
        EXCEPT subdirectory files (loaded when
        Claude reads files in that subdir)
```

Three Scopes Compared

Scope	Path	Who Controls	Shared Via	Can Exclude?
Managed Policy	`/Library/Application Support/ClaudeCode/`	IT / DevOps	MDM, Ansible	Never
User Level	`~/.claude/CLAUDE.md`	You	Stays local	Yes
Project Level	`./CLAUDE.md` or `./.claude/CLAUDE.md`	Team	Git	Yes

Key Behaviors

- **Merging, not overriding** — all files combine into one context
- **More specific wins** — project-level beats user-level on conflicts
- **Managed policy always loads** — designed for org compliance
- **Subdirectory files are lazy** — only load when Claude reads files there
- `@path` **imports** — CLAUDE.md can pull in other files:

  ```
  See @README for overview.
  @~/.claude/my-custom-rules.md
  ```

- **Excluding files** — add to `.claude/settings.local.json`:

  ```
  { "claudeMdExcludes": ["**/other-team/CLAUDE.md"] }
  ```

When to Use CLAUDE.md

- Project overview and build commands
- Key architectural decisions
- Git workflow and branch conventions
- Quick-reference info (< 200 lines ideal)
- Personal cross-project preferences (`~/.claude/CLAUDE.md`)

---

2. RULES

What They Are

Rules are modular instruction files that live in `.claude/rules/`. They replace or supplement a monolithic `CLAUDE.md` by splitting instructions into topic-specific files — and optionally scoping them to specific file paths.

Where Rules Live

```
~/.claude/rules/                    ← User-level (all projects)
your-project/.claude/rules/         ← Project-level (shared via g
it)
```

Both are scanned recursively — subdirectories work.

File Tree Example

```
.claude/
├── CLAUDE.md                       ← Main project instructions
└── rules/
    ├── code-style.md               ← Global (no frontmatter)
    ├── testing.md                  ← Global
    ├── security.md                 ← Global
    ├── frontend/
    │   ├── react.md                ← Scoped to src/components/**
    │   └── styling.md              ← Scoped to src/**/*.css
    └── backend/
        ├── api-design.md           ← Scoped to src/api/**
        └── database.md             ← Scoped to src/db/**
```

Two Types of Rules

Global Rules (no frontmatter)

Load at session start. Always in context. Keep lean (< 200 lines total).

```
# Code Style
-Use 2-space indentation
```

```
-Prefer const over let
-Early returns, not nested if/else
```

Path-Scoped Rules (with `paths:` frontmatter)

Load **on demand** — only when Claude reads matching files. No context cost until triggered.

```
---
paths:
  - "src/api/**/*.ts"
  - "server/**/*.ts"
---

# API Standards
-All endpoints return { success, data, timestamp }
-Zod validation on all request bodies
-Return 400 with validation errors, never 500
```

Path Pattern Syntax (Globs)

Pattern	Matches
`**/*.ts`	All .ts files anywhere
`src/components/**/*.tsx`	React components under src/components
`*.md`	Markdown in project root only
`src/**/*.{ts,tsx}`	Both .ts and .tsx under src/
`tests/**/*.test.ts`	Test files under tests/

Note: Patterns are globs, not regex. `**/*.ts` works. `.*\.ts$` does not.

Loading Timeline

```
SESSION START
│
├── Global rules loaded (no paths: frontmatter)
├── User-level rules loaded (~/.claude/rules/)
```

```
│
│   ... session in progress ...
│
├── Claude reads src/components/Button.tsx
│   └── rules/frontend/react.md loads (matches src/components/*
*)
│
├── Claude reads src/api/users.ts
│   └── rules/backend/api-design.md loads (matches src/api/**)
│
├── Claude reads README.md
│   └── (no path-scoped rules match — nothing new loads)
│
└── SESSION END
```

When to Use Rules vs CLAUDE.md

Use This	For This
CLAUDE.md	Project overview, build commands, quick-ref (one page)
Rules (global)	Detailed standards that apply everywhere
Rules (path-scoped)	Domain-specific guidance (React, API, DB patterns)

Best Practices

- **50-100 lines** per rule file
- **Clear names** — `react.md`, `api-design.md`, not `misc.md`
- **No overlap** between rule files (conflicts = unpredictable)
- **Be specific** — "Use PascalCase for components" not "Name things well"
- **Global rules are expensive** — prefer path-scoped when possible

Gotchas

1. Rules are **guidance, not enforcement** — use permissions/hooks to block

2. Path-scoped rules only trigger **on file reads** — if Claude edits without reading, the rule may not be loaded yet
3. Large global rules **cost tokens every session**
4. Rules **cannot restrict** Claude's actions — only inform them

---

3. MEMORY.MD

What It Is

Auto-memory is Claude's per-project note-taking system. Claude writes topic files and maintains an index at `MEMORY.md`. The first 200 lines of `MEMORY.md` load at session start; topic files load on demand.

How It Differs from CLAUDE.md and Rules

	`CLAUDE.md`	Rules	Auto Memory
Who writes it	You	You	Claude
Structure	Hierarchical (walks dirs)	Modular (per topic)	Flat (per project)
Inheritance	Parent dirs apply to children	Path-scoped	None
Loading	Full file at startup	Global: startup / Scoped: on demand	200-line index + on demand
Purpose	Standing instructions	Detailed standards	Learned context

Storage Structure

```
~/.claude/projects/
├── -Users-stevembp17/
│   └── memory/
│       └── MEMORY.md                    ← Sessions from ~/
├── -Users-stevembp17-Documents/
│   └── memory/
│       └── MEMORY.md                    ← Sessions from ~/Docume
```

```
nts/
├── -Users-stevembp17-Documents--ai-projects-ai-orchestrator/
│   └── memory/
│       ├── MEMORY.md                  ← Index (200 lines loade
d)
│       ├── user_steve.md              ← Topic file (on demand)
│       ├── feedback_no_guessing.md    ← Topic file (on demand)
│       └── project_state.md           ← Topic file (on demand)
└── ... (one folder per project/repo)
```

Key Concept: Memory is FLAT, Not Hierarchical

Unlike CLAUDE.md, memory folders do NOT inherit from each other.

```
CLAUDE.md:                          MEMORY.MD:

~/CLAUDE.md ──────────┐             ~/  memory/    (independen
t)
~/Documents/CLAUDE.md ┤ inherited   ~/Documents/   (independen
t)
~/Documents/proj/     ┘             ~/Documents/proj/ (independ
ent)

Parent applies to child             No inheritance. Totally iso
lated.
```

The Unit of Memory = Git Repo Root

All sessions within the **same git repository** share one memory folder, regardless of which subdirectory you work in. Different repos get different memory. Non-git directories get their own folder based on exact path.

What Triggers a Memory Save

Claude does NOT save automatically. **You** trigger saves through conversation:

Strong Triggers (reliably saves)

```
"Remember that we use pnpm, not npm."
"Save to memory: API tests need Redis on port 6379."
"Note that staging is port 3001, not 3000."
"Make a note: all handlers go in src/api/handlers/."
"This is important to remember — always use early returns."
```

Correction Triggers (feedback memories)

```
Claude runs npm install...
You: "We always use pnpm. Remember that."

Claude uses declare -A...
You: "No bash 4+ features on macOS. Use case statements."

Claude guesses a command...
You: "Don't invent commands. Verify first."
```

Confirmation Triggers (validated approaches)

```
Claude submits one bundled PR...
You: "Yeah, the single PR was the right call."

Claude uses ASCII art...
You: "Perfect — always do diagrams like that."
```

Does NOT Trigger Saves

```
"Fix this bug."                    → too transient
"Let me run the tests."            → ephemeral action
"The file is 50 lines long."       → disposable fact
"That's wrong." (no detail)        → too vague
```

Memory File Types

Type	Trigger Phrase	Example
`user`	“I’m a...”, “I prefer...”	Role, tools, preferences
`feedback`	“Don’t do X”, “Always do Y”	Corrections and validated patterns
`project`	“We’re doing X because...”	Goals, deadlines, architecture decisions
`reference`	“Bugs are tracked in...”	Pointers to external systems

Memory File Format

Each topic file uses frontmatter:

```
---
name: shell compatibility
description: macOS shell constraints — no bash 4+ features
type: feedback
---

No bash 4+ features (declare -A, etc.) on macOS.
Use case statements for conditionals.

**Why:** macOS ships bash 3.2, user targets POSIX/zsh compat.
**How to apply:** Any shell script or bash command generation.
```

MEMORY.md is just an index with links:

```
# Memory Index

## User Profile
-[Steve](user_steve.md) — Mac mini, budget-conscious, ASCII dia
grams

## Feedback
-[No guessing](feedback_no_guessing.md) — verify before stating
-[Shell compat](feedback_shell_compat.md) — no bash 4+ features
```

Cross-Project Memory (There Is None)

Auto-memory is per-project only. For instructions that apply everywhere:

```
"I want ALL sessions to know this"  →  ~/.claude/CLAUDE.md
"I want THIS project to learn"      →  auto memory (MEMORY.md)
"I want my TEAM to follow this"     →  ./CLAUDE.md (in git)
```

Cleanup

Old project memory folders can be safely deleted:

```
# Preview what exists
ls ~/.claude/projects/

# Delete old projects you no longer use
rm -rf ~/.claude/projects/-Users-stevembp17-Documents-old-proje
ct
```

They only recreate if you run Claude Code from that path again.

Quick Reference

Full Loading Order (Session Start)

```
┌──────────────────────────────────────────────────────┐
│ 1. Managed policy CLAUDE.md     (always, forced) │
│ 2. Ancestor CLAUDE.md files     (walk up tree)   │
│ 3. Project CLAUDE.md            (working dir)    │
│ 4. User ~/.claude/CLAUDE.md     (personal)       │
│ 5. Global rules (no paths:)     (all projects)   │
│ 6. MEMORY.md index              (first 200 lines)│
├──────────────────────────────────────────────────────┤
│ ON DEMAND (when Claude reads matching files):    │
│ 7. Path-scoped rules            (frontmatter)    │
│ 8. Subdirectory CLAUDE.md       (lazy)           │
```

```
│ 9. Memory topic files            (on demand)      │
└──────────────────────────────────────────────────┘
```

Decision Matrix

I want to...	Use
Set personal prefs for all projects	`~/.claude/CLAUDE.md`
Share team instructions via git	`./CLAUDE.md` or `./.claude/CLAUDE.md`
Scope instructions to file types	`.claude/rules/` with `paths:` frontmatter
Let Claude learn project quirks	Auto memory (say "remember that...")
Enforce org-wide policy	`/Library/Application Support/ClaudeCode/CLAUDE.md`
Split a large CLAUDE.md into topics	`.claude/rules/` directory

Verify What's Loaded

Run `/memory` in any session to see: - Which `CLAUDE.md` files are active - Which rules are loaded - Auto-memory contents - Toggle auto-memory on/off